1. **Why is floating value not considered in switch cases of C programming language?**

Floating-point values are not allowed in switch cases in C because switch requires constant integral expressions (like int, char, or enum) for case labels. Floating-point numbers are not precise constants due to rounding and representation errors, making them unreliable for exact matching, which is essential in switch statements.

2. **Is it necessary to give break after every switch cases? Explain briefly with appropriate example.**

No, it is not necessary to give a break after every case in a switch statement, but it is usually recommended. Without break, the control "falls through" to the next case, executing it even if it doesn't match, which can cause unintended behaviour unless fall-through is desired.

Example:

```
#include <stdio.h>
int main() {
    int num = 2;
    switch (num) {
        case 1:
            printf("One\n");
            break;
        case 2:
            printf("Two\n");   // No break here
        case 3:
            printf("Three\n");
            break;
        default:
            printf("Other\n");
    }
    return 0;
}
```

Output:

```
Two
Three
```

3. **Differentiate break statement and continue statement with appropriate examples.**

The break and continue statements are used to control loop execution but serve different purposes.

Feature	Break	Continue
Purpose	Exits the loop or switch completely	Skips the current iteration and continues with the next
Use in loops	Terminates the loop early	Skips to the next iteration without ending the loop
Use in switch	Commonly used to exit from a switch case	Not used in switch statements
Control Flow	Transfers control to the statement after the loop	Transfers control to the loop condition (or increment step)

4. **Why is loop controlled structure needed? Using loop controlled structure show the first 10 even numbers.**

Why loop-controlled structure is needed: Loop-controlled structures are essential in programming because they allow repeated execution of a set of statements based on a condition. This reduces code repetition, increases efficiency, and makes the program easier to read and maintain.

Uses of loop-controlled structures:

- Automate repetitive tasks
- Process data in sequences or arrays
- Perform calculations repeatedly under certain conditions

Example: Program to print the first 10 even numbers using a for loop:

```
#include <stdio.h>

int main() {
    int count = 0;
    for(int i = 1; count < 10; i++) {
        if(i % 2 == 0) {
            printf("%d ", i);
            count++;
        }
    }
    return 0;
}
```

Output:

```
2  4 6 8 10 12 14 16 18 20
```

5. **Applying nested loop control structure write a C program to calculate the sum of an arithmetic progression (AP) where the range and common difference will be taken as input.**

```c
#include <stdio.h>

int main() {
    int range, diff, sum = 0, term;

    // Input number of terms and common difference
    printf("Enter the number of terms in the AP: ");
    scanf("%d", &range);

    printf("Enter the common difference: ");
    scanf("%d", &diff);

    int a = 1; // First term of the AP

    // Outer loop (just one iteration here, can be expanded for multiple
APs)
    for (int i = 0; i < 1; i++) {
        sum = 0; // Reset sum for this AP series

        // Inner loop to calculate sum of AP
        for (int j = 0; j < range; j++) {
            term = a + j * diff;
            sum += term;
        }

        printf("Sum of the AP is: %d\n", sum);
    }

    return 0;
}
```

Output:

```
Enter the number of terms in the AP: 5
Enter the common difference: 2
Sum of the AP is: 25
```

6. Applying for loop control loop structure write a C program to generate Fibonacci series where the range will be taken as input.

```c
#include <stdio.h>

int main() {
    int range, first = 0, second = 1, next;

    printf("Enter the number of terms: ");
    scanf("%d", &range);

    printf("Fibonacci Series: ");

    for (int i = 0; i < range; i++) {
        if (i <= 1)
            next = i;       // First two terms are 0 and 1
```

```
        else {
            next = first + second;
            first = second;
            second = next;
        }
        printf("%d ", next);
    }

    return 0;
}
```

Output:

```
Enter the number of terms: 7
Fibonacci Series: 0 1 1 2 3 5 8
```

7. **Define exit controlled loop. Differentiate entry controlled loop and exit controlled loop by analysing control flows of such loops with a suitable example.**

Exit Controlled Loop: An exit-controlled loop checks the loop condition *after* executing the loop body. This ensures that the loop body runs at least once, regardless of whether the condition is initially true or false. In C, the do...while loop is an exit-controlled loop.

Feature	**Entry Controlled Loop (for, while)**	**Exit Controlled Loop (do...while)**
Condition Check	Before entering the loop body	After executing the loop body
Execution Guarantee	May not execute even once	Executes at least once
Common Usage	When number of iterations is known or conditional	When loop body must run at least once
Control Flow Location	Top of the loop	Bottom of the loop
Syntax Simplicity	Preferred for counting loops (for, while)	Preferred for menu-driven or retry-based programs

8. **Applying loop control structure write a C program to check whether an inputted positive integer (greater than 2) by user is prime or not.**

```
#include <stdio.h>

int main() {
    int num, isPrime = 1; // Assume the number is prime

    printf("Enter a positive integer greater than 2: ");
```

```
    scanf("%d", &num);

    if (num <= 2) {
        printf("Invalid input! Number must be greater than 2.\n");
        return 0;
    }

    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            isPrime = 0; // Not a prime number
            break;
        }
    }

    if (isPrime)
        printf("%d is a prime number.\n", num);
    else
        printf("%d is not a prime number.\n", num);

    return 0;
}
```

Output:

```
Enter a positive integer greater than 2: 13
13 is a prime number.

Enter a positive integer greater than 2: 20
20 s not a prime number.
```

9. Why is for loop known as definite loop? Using this loop show the sum of first 10 prime numbers.

```
#include <stdio.h>

int isPrime(int num) {
    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0)
            return 0; // Not a prime number
    }
    return 1; // Prime number
}

int main() {
    int count = 0, num = 2, sum = 0;

    // Loop to find the first 10 prime numbers
    for (int i = 1; count < 10; i++) {
        if (isPrime(num)) {
            sum += num;
            count++;
        }
        num++;
```

```
    }

    printf("Sum of the first 10 prime numbers: %d\n", sum);
    return 0;
}
```

Output:

```
Sum of the first 10 prime numbers: 129
```

10. Using loop control structure how does a C program to show Armstrong number within a given range (in positive integers) which is given user?

```
#include <stdio.h>
#include <math.h>

int main() {
    int start, end, num, originalNum, remainder, n, result;

    // Input range from the user
    printf("Enter the range (start and end): ");
    scanf("%d %d", &start, &end);

    printf("Armstrong numbers between %d and %d are: \n", start, end);

    // Loop through each number in the range
    for (num = start; num <= end; num++) {
        originalNum = num;
        n = 0;
        result = 0;

        // Calculate the number of digits in the number
        while (originalNum != 0) {
            originalNum /= 10;
            ++n;
        }

        originalNum = num;  // Reset originalNum

        // Calculate the sum of the nth powers of the digits
        while (originalNum != 0) {
            remainder = originalNum % 10;
            result += pow(remainder, n);  // sum of nth power of digits
            originalNum /= 10;
        }

        // Check if the sum equals the original number
        if (result == num) {
            printf("%d ", num);
        }
    }

    return 0;
}
```

Output:

```
Enter the range (start and end): 100 999
Armstrong numbers between 100 and 999 are:
153 0 371 407
```

11. How do you to calculate the GCD of three positive integers taken as input?

Algorithm:

1. Use the Euclidean algorithm to find the GCD of two numbers. The Euclidean algorithm states that the GCD of two numbers a and b can be found as:

GCD(a,b)=GCD(b,a%b)GCD(a, b) = GCD(b, a \% b)GCD(a,b)=GCD(b,a%b)

where % is the modulus operator (remainder).

2. Apply the algorithm to three numbers by first finding the GCD of the first two, then using that result to find the GCD with the third number.

C Program to Calculate GCD of Three Numbers:

```
#include <stdio.h>

// Function to calculate GCD of two numbers using Euclidean algorithm
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    int num1, num2, num3, result;

    // Input three numbers from the user
    printf("Enter three positive integers: ");
    scanf("%d %d %d", &num1, &num2, &num3);

    // First, find GCD of num1 and num2
    result = gcd(num1, num2);

    // Then, find GCD of result and num3
    result = gcd(result, num3);

    // Output the result
    printf("GCD of %d, %d, and %d is: %d\n", num1, num2, num3, result);

    return 0;
}
```

Output:

```
Enter three positive integers: 24 36 60
GCD of 24, 36, and 60 is: 12
```

12. **Why do-while loop is named as "post-test"? Define infinite loop with examples using pre-test loop**.

The **do-while loop** is called a **post-test loop** because it **checks the condition *after* executing the loop body**. This ensures the loop body executes at least once, regardless of whether the condition is initially true or false.

Definition of Infinite Loop: An **infinite loop** is a loop that **never terminates** on its own because the **loop condition always remains true** (or there's no condition to break it).

Examples of Infinite Loops using Pre-Test Loops:

1. **Using while loop (pre-test):**

```
while (1) {
    printf("This is an infinite loop.\n");
}
```

2. **Using for loop (pre-test):**

```
for (;;) {
    printf("This is also an infinite loop.\n");
}
```

13. **Applying for loop write a C program to evaluate the following summation series where the range will be taken as input from the keyboard.**

$$1+\frac{x}{2!}+\frac{x}{3!}+\frac{x}{4!}+\cdots$$

```
#include <stdio.h>

// Function to calculate factorial of a number
int factorial(int num) {
    int fact = 1;
    for (int i = 2; i <= num; i++) {
        fact *= i;
    }
    return fact;
}

int main() {
    int x, n;
    float sum = 1.0; // Starting from 1

    // Input x and range n
    printf("Enter the value of x: ");
    scanf("%d", &x);
    printf("Enter the range (n): ");
    scanf("%d", &n);
```

```
    // Compute the series sum
    for (int i = 2; i <= n; i++) {
        sum += (float)x / factorial(i);
    }

    // Output the result
    printf("Sum of the series is: %.4f\n", sum);

    return 0;
}
```

Output:

```
Enter the value of x: 2
Enter the range (n): 5
Sum of the series is: 1.7667
```

14. Applying for loop write a C program to find the sum of following summation series where the range will be taken as input from the keyboard.

$$1+\frac{2^2}{2!}+\frac{3^3}{3!}+\frac{4^4}{4!}+\cdots$$

```
#include <stdio.h>
#include <math.h>

// Function to calculate factorial
long long factorial(int num) {
    long long fact = 1;
    for (int i = 2; i <= num; i++) {
        fact *= i;
    }
    return fact;
}

int main() {
    int n;
    double sum = 1.0; // Start with first term as 1

    // Input range from user
    printf("Enter the range (n): ");
    scanf("%d", &n);

    // Loop to calculate and add each term
    for (int i = 2; i <= n; i++) {
        double numerator = pow(i, i);
        double denominator = factorial(i);
        sum += numerator / denominator;
    }

    // Output the result
    printf("Sum of the series is: %.4lf\n", sum);
```

```
    return 0;
}
```

Output:

```
Enter the range (n): 4
Sum of the series is: 20.3333
```

15. How do you display following pattern using loop control structure in C programming?

1
2
3 4
4 5 6
5 6 7 8

```
#include <stdio.h>
int main() {
    int rows = 5;

    // Outer loop for rows
    for (int i = 1; i <= rows; i++) {
        int value = i;

        // Inner loop for columns (i values in each row)
        for (int j = 1; j <= i; j++) {
            printf("%d\t", value);
            value++; // Increment value for next column
        }

        printf("\n"); // Newline after each row
    }

    return 0;
}
```

16. Using for loop write a C program to print the following pattern up to a positive inputted integer (greater than 1).

1
1 3 5
1 3 5 7 9

```
#include <stdio.h>

int main() {
    int rows;

    // Input the number of rows
```

```
    printf("Enter a positive integer greater than 1: ");
    scanf("%d", &rows);

    if (rows <= 1) {
        printf("Input must be greater than 1.\n");
        return 1;
    }

    // Outer loop for each row
    for (int i = 1; i <= rows; i++) {

        // Print leading tabs to center the pattern
        for (int space = 1; space <= rows - i; space++) {
            printf("\t");
        }

        // Print odd numbers (2*i - 1 times)
        int num = 1;
        for (int j = 1; j <= 2 * i - 1; j++) {
            printf("%d\t", num);
            num += 2; // Move to next odd number
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter a positive integer greater than 1: 3
        1
    1   3   5
1    3   5   7   9
```

17. Using loop control structure write a C program to print Armstrong number within a given range by user.

```
#include <stdio.h>
#include <math.h>

int main() {
    int start, end;

    // Input the range
    printf("Enter the starting number: ");
    scanf("%d", &start);
    printf("Enter the ending number: ");
    scanf("%d", &end);

    printf("Armstrong numbers between %d and %d are:\n", start, end);

    // Loop through each number in the range
```

```
    for (int num = start; num <= end; num++) {
        int originalNum = num;
        int sum = 0;
        int digits = 0;

        // Count number of digits
        int temp = originalNum;
        while (temp != 0) {
            temp /= 10;
            digits++;
        }

        // Calculate sum of digits raised to the power of 'digits'
        temp = originalNum;
        while (temp != 0) {
            int digit = temp % 10;
            sum += pow(digit, digits);
            temp /= 10;
        }

        // Check if it's an Armstrong number
        if (sum == originalNum) {
            printf("%d ", originalNum);
        }
    }

    printf("\n");
    return 0;
}
```

Output:

```
Enter the starting number: 100
Enter the ending number: 999
Armstrong numbers between 100 and 999 are:
153 370 371 407
```

18. Compare if-else and switch in terms of C programming language.

Feature	if-else Statement	switch Statement
Condition Type	Can evaluate multiple relational or logical expressions	Works only with constant integral expressions (e.g., int, char)
Flexibility	More flexible – supports ranges, logical operators, etc.	Less flexible – supports only exact matches
Readability	Becomes complex and less readable for many conditions	More readable for fixed multiple cases
Performance	Slower in case of many conditions, evaluated sequentially	Faster in some cases (like compiler jump tables)

19. Applying iterative approach write a C program to determine the factorial of a positive integer (less than 8) inputted by user.

```c
#include <stdio.h>

int main() {
    int num;
    int factorial = 1;

    // Input from user
    printf("Enter a positive integer less than 8: ");
    scanf("%d", &num);

    // Input validation
    if (num <= 0 || num >= 8) {
        printf("Invalid input. Please enter a number between 1 and
7.\n");
        return 1;
    }

    // Iterative loop to compute factorial
    for (int i = 1; i <= num; i++) {
        factorial *= i;
    }

    // Output the result
    printf("Factorial of %d is %d\n", num, factorial);

    return 0;
}
```

Output:

```
Enter a positive integer less than 8: 5
Factorial of 5 is 120
```

20. Applying an iterative approach, write a C program to check whether the given number is Palindrome or not.

```c
#include <stdio.h>

int main() {
   int num, reversed = 0, original, remainder;

   // Input from user
   printf("Enter a positive integer: ");
   scanf("%d", &num);

   original = num;  // Store original number

   // Iterative loop to reverse the number
   while (num > 0) {
      remainder = num % 10;
      reversed = reversed * 10 + remainder;
```

```
        num = num / 10;
    }

    // Check if the reversed number is equal to the original
    if (original == reversed) {
        printf("%d is a Palindrome number.\n", original);
    } else {
        printf("%d is NOT a Palindrome number.\n", original);
    }

    return 0;
}
```

Output:
Enter a positive integer: 1221
1221a Palindrome number.

21. Applying nested loop structure in C programming write a C program to generate following pattern up to a line number (which is a positive integer given by the user.)

```
                1
            0   1
        1   0   1
    0   1   0   1
```

```
#include <stdio.h>

int main() {
    int rows;

    // Input from user
    printf("Enter number of lines (positive integer): ");
    scanf("%d", &rows);

    // Outer loop for rows
    for (int i = 0; i < rows; i++) {

        // Print leading tabs for triangle shape
        for (int space = 1; space <= rows - i - 1; space++) {
            printf("\t");
        }

        // Print alternating 1s and 0s
        int val = i % 2; // Start with 0 for even row, 1 for odd row

        for (int j = 0; j <= i; j++) {
```

```
            printf("%d\t", val);
            val = 1 - val; // Alternate between 1 and 0
        }

        printf("\n"); // Newline after each row
    }

    return 0;
}
```

Output:

```
Enter number of lines (positive integer): 4
                  1
            0     1
      1     0     1
 0  1  0     1
```

22. Using loop control structure, how do you print (through C programming code) the following pattern up to a positive inputted integer (greater than 1)?

```
            1
      1     3     5
1     3     5     7     9
```

```
#include <stdio.h>

int main() {
    int rows;

    // Input from user
    printf("Enter a positive integer greater than 1: ");
    scanf("%d", &rows);

    if (rows <= 1) {
        printf("Input must be greater than 1.\n");
        return 1;
    }

    // Outer loop for each row
    for (int i = 1; i <= rows; i++) {

        // Print leading tabs to shape the triangle
        for (int space = 1; space <= rows - i; space++) {
            printf("\t");
        }

        // Print odd numbers: 2*i - 1 terms
        int odd = 1;
        for (int j = 1; j <= 2 * i - 1; j++) {
            printf("%d\t", odd);
            odd += 2;
```

```
        }

        printf("\n"); // Move to next line
    }

    return 0;
}
```

Output:

```
Enter a positive integer greater than 1: 3
        1
    1   3   5
1   3   5   7   9
```

MCQs

1. The output of this code snippet is :

```
void main()
{
    int a=2;
    switch(a)
    {
        printf("HELLO");
        case 1: printf("1"); break;
        case 2: printf("2"); break;
    }
}
```

Answer: b) HELLO2

2. What will be the output?

```
void main()
{
    int a=2;
    switch(a)
    {
        case 1: printf("1");
        case 2: printf("2");
    }
}
```

Answer: b) 2

3. What will be the output?

```
void main()
{
    float b=2.7;
    switch(b)
    {
        case 1: printf("1");
        case 2: printf("2");
```

```
    }
}
```

Answer: d) Compilation Error

4. The output is :

```
void main()
{
    char c=66;
    switch(c)
    {
        case 67: printf("A");
        case 66: printf("B");
    }
}

}
```

Answer: c) B

5. The value of the variable 'a' is :

```
void main()
{
    int i,a;
    for(i=0;i<=5;i++);
    {a=i;}
}
```

Answer: b) 6

6. Which will be the output?

```
void main ()
{
    int i, j=3;
    for(i=0; i<j; i++) printf("%d %d ", i, j);
}
```

Answer: b) 0 3 1 3 2 3

7. Which will be the output?

```
void main()
{
    int i='0';
    switch(i)
    {
        case 48: printf ("one");
        case 0: printf ("two");
        case 49: printf ("three");
        default: printf ("error");
    }
}
```

Answer: a) one two three error

8. Which will be the output?

```
void main ()
{
    int i, j;
    for(i=0,j=5;i<j;i++);
    printf("%d %d", i, j);
}
```

Answer: a) 5 5

9. The output of the code will be .

```
int main()
{
    int i = 3, j = 4;
    switch(i || j)
    {
        case 1: printf("inside case 1"); break;
        case 3: printf("inside case 3"); break;
        case 4: printf("inside case 4"); break;
        case 0: printf("inside case 7"); break;
    }
    return 0;
}
```

Answer: a) inside case 1

10. What will be the output?

```
void main()
{
    char i;
    for (i='2'; i>='0'; i--)
        printf("%d ", i);
}
```

Answer: b) 50 49 48

11. Post-test loop refers to:

Answer: c) do-while loop